

Webforge Studio — Debugging Report

Audit of issues found during local testing | June 26, 2026

Overview

Three bugs were identified when running the Webforge Studio desktop app locally. All three have been fixed and verified. This report explains what went wrong, why it happened, and how each issue was resolved.

Issue 1: White / blank dashboard window

Symptom

When launching the app, the main dashboard window appeared completely blank (white page). No UI elements, tabs, or text were visible.

Root Cause

The binary was built via `cargo build` directly, which embeds the frontend `dist/` directory at compile time through Tauri's `generate_context!()` macro. However, the `dist/` folder was rebuilt (via `npm run build`) AFTER the Rust binary was compiled. This meant the binary contained stale or empty frontend assets. When the webview tried to load the bundled JavaScript, it got 404s for the asset files, resulting in a blank page with no JS executing.

Fix

- Run `npx tauri dev` instead of building manually — this orchestrates both the frontend build and Rust compilation in the correct order.
- When building a production binary, use `npx tauri build` which runs `beforeBuildCommand` (`npm run build`) before compiling Rust.
- Added diagnostic logging in the Rust setup hook that prints the webview URL and embedded server port to `stderr` for debugging.

Issue 2: "Failed to open workspace: missing field folderPath"

Symptom

When clicking the **Open** button on a workspace card, the app showed an alert: *Failed to open workspace: invalid args `workspace` for command `open\_workspace`: missing field `folderPath`*

Root Cause

The Rust `Workspace` struct uses `#[serde(rename_all = "camelCase")]`, so it expects JSON fields named `folderPath`, `startPage`, and `serverUrl`. However, the TypeScript frontend read workspace rows from the SQLite database, which stores columns in `snake_case` (`folder_path`, `start_page`, `server_url`). The frontend passed the raw DB row directly to `invoke("open_workspace", { workspace: ws })` without converting the field names to `camelCase`. Serde rejected the payload because the expected `camelCase`

fields were missing.

Fix

Added a `toCamelWorkspace()` helper function in `src/main.ts` that maps the `snake_case` DB fields to the `camelCase` shape expected by Rust. Applied this conversion at both call sites: the `openWorkspace()` click handler and the `autoResumeWorkspaces()` launch handler.

Code

```
function toCamelWorkspace(ws: Workspace) {
  return {
    id: ws.id, name: ws.name, slug: ws.slug, kind: ws.kind,
    folderPath: ws.folder_path,
    startPage: ws.start_page,
    serverUrl: ws.server_url,
  };
}
```

Issue 3: "not found: index.html" in editor + @@include directives visible

Symptom

Two related problems appeared in the editor view:

- (a) The editor iframe showed *not found: index.html* when opening a workspace whose folder had been deleted (it was in a temporary directory that got cleaned up).
- (b) The file manager listed demo pages containing raw `@@include(...)` template directives, which rendered as broken pages with literal `@@include` text instead of resolved HTML.

Root Cause (a): Missing folder

The `open_workspace` Rust command registered the workspace's folder path with the embedded HTTP server without checking if the folder actually existed on disk. When the folder was missing (e.g., a temp directory that was cleaned up), the server silently registered a non-existent path. When the editor iframe requested `/site/<id>/index.html`, the server looked in the missing folder and returned *not found*.

Fix (a)

Added a folder existence check in `open_workspace` (in `src-tauri/src/workspace.rs`). If the folder doesn't exist, the command returns a clear error message explaining the situation and suggesting the user create a new managed workspace instead of a cryptic *not found* in the iframe.

Root Cause (b): @@include templates

The bundled Webforge editor includes two copies of the demo site:

- demo/landing/*.html** — compiled pages with all `@@include` directives resolved
- demo/landing/src/*.html** — source templates with unresolved `@@include` directives

The original VVVEB.js used PHP's `editor.php` which only scanned specific directories for the file manager. The Rust-native replacement (`collect_pages` in `server.rs`) recursively scanned the entire `demo/` tree, including the `src/` subdirectory. This caused source templates with raw `@@include` directives to appear in the file manager and render as broken pages when clicked.

Fix (b)

Modified `walk_html_pages()` in `server.rs` to skip any subdirectory named `src` during the recursive scan. This excludes all unresolved source templates from the file manager. Only the compiled demo pages (which have no `@@include` directives) now appear. Added a unit test (`collect_pages_excludes_src_dir`) to verify the exclusion.

Summary of Changes

Issue	File(s) Modified	Change
1. Blank dashboard (stale dist)	<code>src-tauri/src/lib.rs</code>	Added setup logging; use tauri dev/build for correct build order
2. Missing folderPath (serde mismatch)	<code>src/main.ts</code>	Added <code>toCamelWorkspace()</code> helper; applied at 2 call sites
3a. not found: <code>index.html</code> (missing folder)	<code>src-tauri/src/workspace.rs</code>	Added folder existence check in <code>open_workspace</code>
3b. <code>@@include</code> directives (src templates shown)	<code>src-tauri/src/server.rs</code>	<code>walk_html_pages</code> skips <code>src/</code> dirs; added exclusion test

Test Results

All 29 Rust unit tests pass (23 server tests + 6 workspace tests). TypeScript typecheck passes with no errors. The app launches and renders the dashboard correctly. Managed workspaces can be created and opened in the editor.